

In this case, you can display data from the related model by connecting field names using . (dot).

For example, when you want to display the `div_name` field through the *Employee* model, you can describe it like `{{ object.division.div_name }}`. In this case, the object represents a record object in the *Employee* model.

Below is an example of handling the template file for the model with `ForeignKey`.

Example:

 **employee_detail.html**

```
<h1>ForeignKey Test</h1>

<p>{{ object.name }}</p>
<p>{{ object.priority }}</p>
<p>{{ object.reg_date }}</p>
<p>{{ object.division.div_name }}</p>
<p>{{ object.division.in_scope }}</p>
```

Note: also updated the following files

- **views.py** : added EmployeeDetail view
- **urls.py** : added a URL pattern for the view

 In browser

ForeignKey Test

Eleanor Pena

M

March 30, 2023

Product Development

True

Model with ManyToManyField

Showing data from a different model connected through `ManyToManyField` is slightly more complex. As the connected model may have multiple record objects, you need to iterate to deliver all record objects. For example, the Web Application Basic course may have multiple employees assigned.

To display `ManyToMany` Field, you need to do two things

1. Use the `all()` method to call the `employee` object as a `QuerySet`
2. Use the `for` statement